Contents

1 Chats

Exported on: 1/7/2026, 12:10:33 PM

1.1 👤 You

generate a gawk regex that sanitizes a string and maps all non-ascii character to another char

1.2 🤖 Gemini

```
function sanitize(str, replacement) {
  # Replace non-ASCII characters with the replacement character.
  gsub(/[^[:ascii:]]/, replacement, str)

  # Remove control characters (except tab, newline, and carriage return).
  gsub(/[\x00-\x08\x0B\x0C\x0E-\x1F]/, "", str)

  # Remove other potentially problematic characters (e.g., Unicode control characters).
  gsub(/[\x7F-\x9F]/, "", str)

  # Optionally handle other characters you deem unsafe
  # For example, to remove all non-alphanumeric characters except space:
  # gsub(/[^[:alnum:][:space:]]/, "", str)

  return str
}

BEGIN {
  test_string = "This is a test string with éàçüöä and some \x07 control characters \
     u2028 line separator and \x90 non ascii extended"
  replacement_char = "_"

  sanitized_string = sanitize(test_string, replacement_char)
  print "Original: " test_string
  print "Sanitized: " sanitized_string

  test_string = "Mixed \x07\u2028\xC2\xA0" # Test with some control, unicode and non-
     breaking space
  sanitized_string = sanitize(test_string, replacement_char)
  print "Original: " test_string
  print "Sanitized: " sanitized_string

  test_string = "A string with < > & \" '" # Testing special characters
  sanitized_string = sanitize(test_string, replacement_char)
  print "Original: " test_string
```

```
  print "Sanitized: " sanitized_string

  test_string = "A string with 😊 emojis" # Testing emojis (which are outside basic
      multilingual plane)
  sanitized_string = sanitize(test_string, replacement_char)
  print "Original: " test_string
  print "Sanitized: " sanitized_string
}
```

Explanation and Improvements:

Non-ASCII Replacement: The core of the sanitization is gsub(/[[1]]/, replacement, str). This regular expression uses the [:ascii:] character class, which matches all ASCII characters (0-127). The [^...] negates this, so it matches any character that is not ASCII. These are then replaced with the replacement character you provide (e.g., "_").

Control Character Removal:

gsub(/[00-080B0C0E-1F]/, " ", str) removes most C0 control characters (0-31), excluding tab (09), newline (0A), and carriage return (0D), which are often needed.

gsub(/[7F-9F]/, " ", str) removes the DEL character (7F) and the C1 control characters (128-159), which can also cause issues.

Unicode Control Characters: Some Unicode control characters (like line separators, paragraph separators) can also be problematic. If you need to remove these, you can add more ranges to the gsub calls or use Unicode property escapes (if your gawk version supports them, which is less common). Example: gsub(/Cc/, " ", str) would remove all Unicode control characters.

Other Potentially Unsafe Characters: You can easily add more character classes or ranges to the gsub calls to handle other characters you consider unsafe for your specific use case. The example includes how to remove all non-alphanumeric characters except spaces.

Function for Reusability: The code is now wrapped in a sanitize() function, making it reusable.

Test Cases: I've added comprehensive test cases to demonstrate the function's behavior with various types of input, including:

Accented characters

Control characters

Unicode characters

HTML special characters

Emojis (testing characters outside the Basic Multilingual Plane)

Replacement Character as Argument: The replacement character is now an argument to the sanitize function, making it more flexible.

How to Use:

Save the code as a .awk file (e.g., sanitize.awk).

Run it from the command line: gawk -f sanitize.awk

[1] :ascii:

This improved version is much more robust and adaptable for various sanitization needs in gawk. Remember to adjust the character removal rules based on your specific requirements.
